

Prep Manager - Orchestrator Integration Guide

Technical Interface Specification | Core Adapter Endpoints | Evidence Record Schema | Operational SLAs

Executive Purpose: This guide provides orchestrators (e.g. Saif's Master Orchestrator) with the authoritative contract to invoke inbound Amazon FBA prep compliance checks via the **Prep Manager Agent (Pod 02)**. It details the endpoint specification, payload formats, deterministic evidence records, and failure handling guarantees.

1. Specification vs. Implementation

A strict architectural boundary separates the cross-pod specification from Pod 02's internal implementation:

Dimension	Specification Requirement (The Role)	Agent Implementation (Pod 02 Solution)
Pipeline Position	Step 02 in 5-pod inbound chain: Receiving (01) → Prep (02) → Pack (03) → Manifest (04) → Recovery (05).	Accepts items post-arrival, inspects 3 camera angles (front, back, label), and emits machine-verifiable evidence records.
Shared Identifier	Strict join on shared foreign key unit_id (UNIT-0001 through UNIT-0100).	All records, photographic vectors, and telemetry key deterministically on unit_id and org_id tenant.
Thin Adapter Mandate	Agent must be a pure evaluation service. Multi-station routing, retries, and recovery claims belong to the orchestrator.	Implemented as a stateless POST /run adapter. Contains zero cross-pod workflow routing, maintaining clean separation.
Inspection Engine	Verify FBA prep: polybag seals, suffocation warning, FNSKU placement, barcode coverage, expiry date, handling marks.	Hardware-accelerated ONNX Nano detector (DirectML/GPU/CPU) + deterministic OpenCV spatial engine + Amazon FBA rule engine.
Unit Economics	Prep verification must fit prep fees (\$0.40 - \$1.10) with negligible compute overhead.	Delivers sub-cent compute (~\$0.00015 per inspection check), preserving >99.9% of merchant operating margin.

2. The API Endpoint

The service provides both the standardized **Round 3 Agent Adapter** (/run) for automated orchestration and high-performance direct endpoints for camera stations:

Endpoint	Method	Protocol / Payload	Primary Use Case & Description
/run	POST	application/json	Primary Orchestrator Adapter. Conforms to Round 3 Agent I/O spec. Accepts image refs, runs inspection, and returns verdict with full evidence.
/api/v1/inspect	POST	application/json	Synchronous direct inspection endpoint using storage image references.
/api/v1/inspect/stream	POST	text/event-stream (SSE)	Reference-based progressive streaming across 5 milestones (front → back → label → done).
/api/v1/inspect/stream/upload	POST	multipart/form-data	Multipart direct camera image upload (front, back, label) with live SSE progress stream.
/health /health/ready	GET	application/json	Liveness & readiness probes verifying ONNX runtime, detector model weights, and storage.
/api/v1/records/{unit_id}	GET	application/json	Retrieves stored historical evidence record with tenant row-level security isolation.

Base URLs: Local: http://localhost:8000 | Container/Cloud: Configured via BACKEND_URL environment variable.

Required Request Headers:

- Content-Type: application/json
- X-Org-ID: org_demo_alpha or org_demo_bravo (Mandatory multi-tenancy header | Rule 1)
- X-API-Key: <SERVER_API_KEY> or Authorization: Bearer <TOKEN> (When REQUIRE_AUTH=true)

3. How to Send a Request

To invoke an inspection, send a POST /run request containing the unit identifier, multi-angle camera inputs, and specific work order rules. The adapter is resilient: images can be supplied as local file paths, HTTP URLs, or base64 strings.

Input Request Parameters (POST /run):

Parameter	Type	Required?	Description & Example Value
task_id	string	Optional	Unique orchestrator execution ID. Auto-generated if omitted (e.g. "task-prep-0912-abc").
unit_id	string	Required	Foreign key identifier joining all 5 pods (e.g. "UNIT-0001").
org_id	string	Required	Tenant organization identifier: "org_demo_alpha" or "org_demo_bravo".
image_refs	object	Required*	Dict containing "front", "back", "label" image paths, URLs, or base64 strings.
work_order	object	Optional	Prep flags: wo_polybag (bool), wo_suffocation_warning (bool), wo_expiry_date (bool), wo_handling_marks (str).
force_reinspect	boolean	Optional	Defaults to false. Set true to bypass duplicate cache and force re-evaluation.

Complete Request JSON Payload Example:

```
{
  "task_id": "orchestrator-task-0912-abc",
  "unit_id": "UNIT-0001",
  "org_id": "org_demo_alpha",
  "image_refs": {
    "front": "cube_prep_dataset/images/UNIT-0001_front.jpg",
    "back": "cube_prep_dataset/images/UNIT-0001_back.jpg",
    "label": "cube_prep_dataset/images/UNIT-0001_label.jpg"
  },
  "work_order": {
    "work_order_id": "WO-3000",
    "fba_shipment_id": "FBA-CUBE-100",
    "sku": "SKU-CABLE-USB-C",
    "asin": "B0DUMMY261",
    "fnsku": "X00CUBE0001",
    "wo_polybag": false,
    "wo_suffocation_warning": false,
    "wo_expiry_date": false,
    "wo_handling_marks": "fragile",
    "prep_price_usd": 0.75
  },
  "force_reinspect": false
}
```

Ready-to-Use Orchestrator Code Snippets:

cURL Command	Python (httpx / requests)
<pre>curl -X POST http://localhost:8000/run \ -H "Content-Type: application/json" \ -H "X-Org-ID: org_demo_alpha" \ -d '{ "task_id": "task-01", "unit_id": "UNIT-0001", "org_id": "org_demo_alpha", "image_refs": { "front": "images/UNIT-0001_front.jpg", "back": "images/UNIT-0001_back.jpg", "label": "images/UNIT-0001_label.jpg" }, "work_order": { "wo_polybag": true, "wo_suffocation_warning": true } }'</pre>	<pre>import httpx payload = { "task_id": "task-01", "unit_id": "UNIT-0001", "org_id": "org_demo_alpha", "image_refs": { "front": "images/UNIT-0001_front.jpg", "back": "images/UNIT-0001_back.jpg", "label": "images/UNIT-0001_label.jpg" }, "work_order": {"wo_polybag": True} } resp = httpx.post("http://localhost:8000/run", json=payload, headers={"X-Org-ID": "org_demo_alpha"}, timeout=10.0) res = resp.json() print("Verdict:", res["verdict"])</pre>

4. The Response (Evidence Record) - Expected JSON Output

The response delivers a deterministic compliance verdict conforming strictly to prep_evidence_contract.json. It contains high-level orchestrator fields, itemized checks, spatial evidence vectors, and the complete audit record:

```
{
  "task_id": "orchestrator-task-0912-abc",
  "agent": "prep",
  "stage": "prep",
  "org_id": "org_demo_alpha",
  "status": "success",
  "decision": "PASS",
  "verdict": "PASS",
  "confidence": 0.96,
  "explanation": "All applicable visual preparation checks are supported by the available evidence.",
  "checks": {
    "polybag_present_sealed": { "verdict": "PASS", "applicable": false, "detail": "Polybag not required by work order." },
    "suffocation_warning": { "verdict": "PASS", "applicable": false, "detail": "Suffocation warning not required." },
    "fnsku_label_placement": { "verdict": "PASS", "applicable": true, "detail": "FNSKU is flat, clear of seams & edges." },
    "original_barcode_covered": { "verdict": "PASS", "applicable": true, "detail": "Original manufacturer barcode obscured." },
    "expiry_date": { "verdict": "PASS", "applicable": false, "detail": "Expiry date verification not required." },
    "handling_marks": { "verdict": "PASS", "applicable": true, "detail": "All required handling marks present." }
  },
  "evidence": {
    "image_quality": { "front_blur": 1175.01, "back_blur": 609.09, "label_blur": 1448.26, "min_blur": 609.09, "is_ambiguous": false },
    "package_bounds": [148, 75, 475, 385],
    "fnsku": { "detected": true, "placement": "flat", "box": [425, 196, 160, 115], "edge_distance_px": 38, "seam_overlap_iou": 0.0 }
  },
  "record": {
    "record_id": "PRP-0001",
    "unit_id": "UNIT-0001",
    "org_id": "org_demo_alpha",
    "overall_status": "PASS",
    "workflow_state": "completed",
    "calibration": { "is_calibrated": true, "front_quality": { "blur_metric": 1175.01, "glare_ratio": 0.0285 } },
    "performance": { "latency_ms": 3135.22, "estimated_compute_cost_usd": 0.00015, "cost_within_economics": true },
    "captured_at": "2026-09-20T08:17:00Z"
  },
  "latency_ms": 3135.22
}
```

Orchestrator Decision Routing Matrix:

Verdict	Meaning	Downstream Orchestrator Routing Action
PASS	All applicable checks verified compliant.	Forward unit to Step 03 Pack Manager . Pass record to Step 05.
FAIL	One or more defects detected (seam overlap, missing warning, etc.).	Divert physical conveyor to Rework / Relabel Station with itemized failure reasons.
UNCERTAIN	Visual ambiguity (blur < 25, glare, missing camera angle).	Route to Human QA Station (Fail-Open guarantee). Never halt warehouse conveyors.

Important Integration Notes

Orchestrator engineers must observe these 6 critical operational rules and integration behaviors:

- **1. Multi-Tenancy Scoping (Rule 1):** Every request must supply org_id (or header X-Org-ID) matching org_demo_alpha or org_demo_bravo. Cross-tenant queries are strictly rejected with 403 Forbidden.
- **2. Fail-Open Safety Wrapper (Rule 3):** If camera captures are blurry (blur score < 25), glared, or missing, the agent does NOT raise 500 errors. It returns UNCERTAIN with workflow_state: pending_review so physical lines never jam.
- **3. Idempotency & Duplicate Prevention:** Calling /run multiple times for the same unit returns the cached record instantly without redundant compute. To force fresh re-inspection, supply "force_reinspect": true.
- **4. Downstream Interoperability (Step 05 Recovery):** The full evidence record in record contains calibrated photographic blur metrics, glare ratios, and pixel coordinates. The Recovery Manager (Pod 05) requires this exact structure to reverse Amazon chargebacks.
- **5. Latency & Batching SLAs:** Single unit evaluation latency is ~3000ms - 4000ms. For large batches (>10 units), submit jobs via POST /api/v1/jobs and poll GET /api/v1/jobs/{job_id} to prevent gateway timeouts.
- **6. Human Overrides are Data (Honesty Rule 2):** When warehouse operators override an agent verdict, call POST /api/v1/override. The original verdict, operator ID, and mandatory explanation are immutably logged into the audit trail.

Integration Readiness: Verified & fully compliant with CUBE 2026 Round 3 Agent I/O Contract.

Technical Support: Pod 02 Prep Compliance Team | Repository: cube26-prp-0211-dhanvind360